Deep Learning

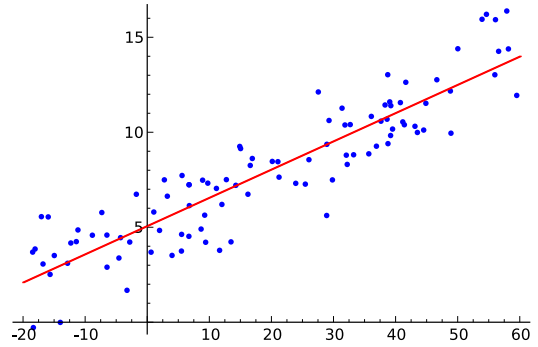
Data Science

Deep Learning

- reminder: linear models
- neural networks
- network training
- advanced network architectures
  - convolutional neural networks
  - transfer learning
  - recurrent neural networks

Reminder: Linear Models

Linear Regression



following Gaussian distribution

data: vector with p different feature values for this example i

$$\mathbb{E}[Y|\boldsymbol{X} = \boldsymbol{x}_i] = f(\boldsymbol{x}_i) = \boldsymbol{w} \cdot \boldsymbol{x}_i + b$$

vector with slope parameters (one for each of the p features, but identical for all examples), aka weights

single intercept parameter, aka bias

training → parameter estimation

Classification: Logistic Regression

binary classification: $y = 1$ or $y = 0$

→ predict probabilities p_i for $y = 1$

- logit (log-odds) as link function
- Y following Bernoulli distribution

still Gaussian linear model

$$\text{logit}(\mathbb{E}[Y|\boldsymbol{X} = \boldsymbol{x}_i]) = \ln\left(\frac{p_i}{1-p_i}\right) = \boldsymbol{w} \cdot \boldsymbol{x}_i + b$$

$$p_i = \frac{1}{1 + e^{-(\boldsymbol{w} \cdot \boldsymbol{x}_i + b)}}$$

Multi-Classification: Softmax

for classification in K categories:

multinomial logistic regression

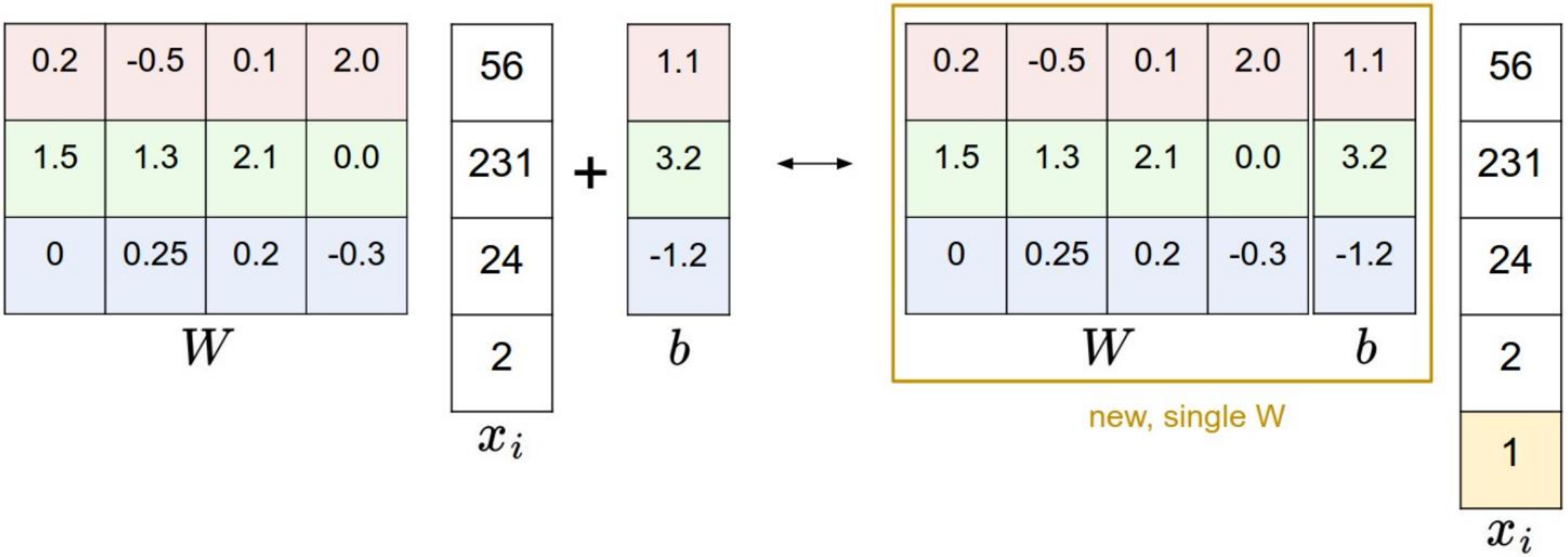
- use "score" function with K outputs

$$f_k(\boldsymbol{x}_i) = \boldsymbol{w}_k \cdot \boldsymbol{x}_i + b_k$$

- and convert into probabilities by means of softmax function

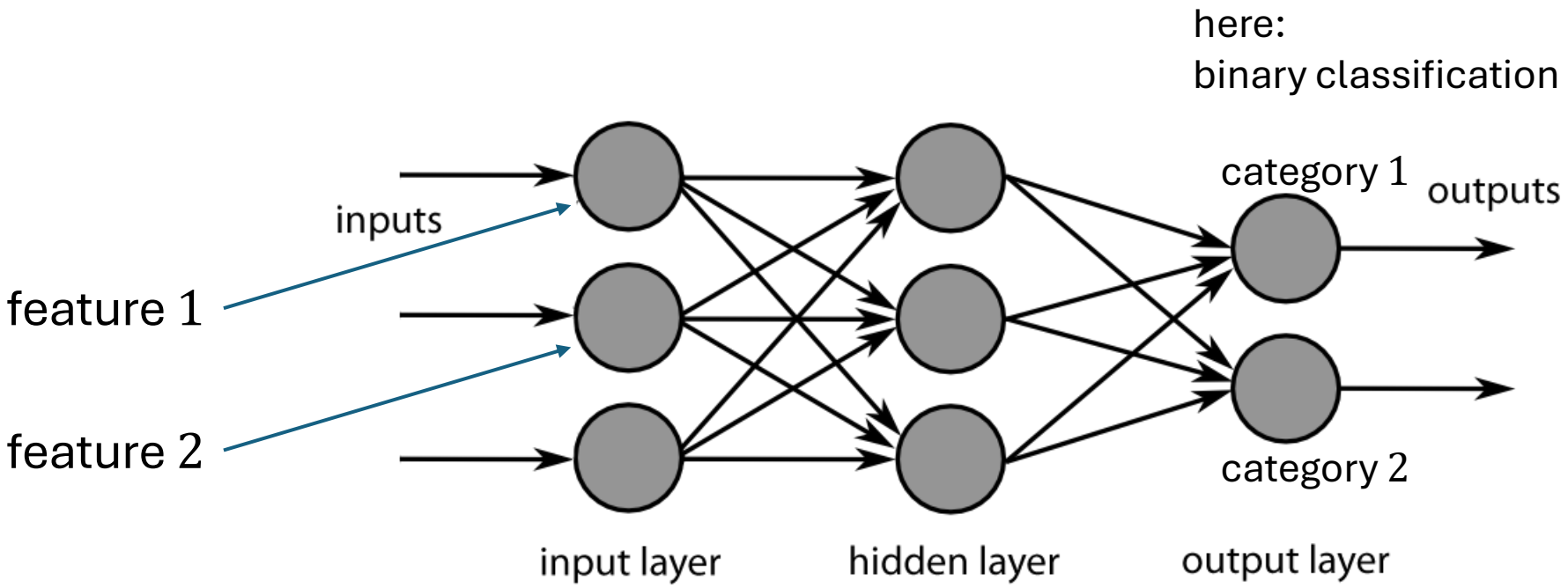
$$\frac{\exp\left(f_k(\boldsymbol{x}_i)\right)}{\sum_{l=1}^{K} \exp\left(f_l(\boldsymbol{x}_i)\right)}$$

Simplified Matrix Multiplication: Bias Trick



Neural Networks:
Combine Many Linear Models

Neural Networks



each node exchanges information only with directly connected nodes
→ parallel computation

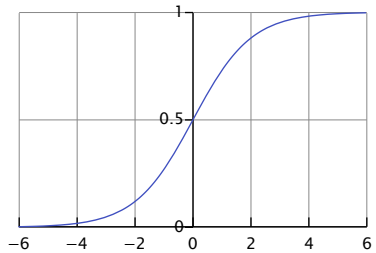
Artificial Neuron

linear model with parameters called weights $\boldsymbol{w}$ (including bias, not shown here for simplicity)

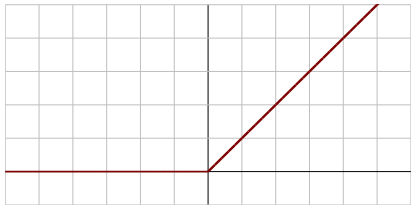
non-linear via (differentiable) activation function on top of linear model

examples:

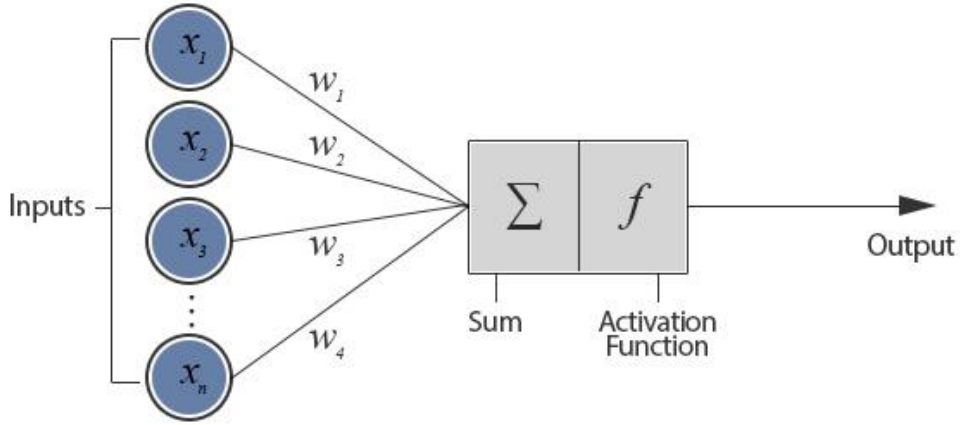
- sigmoid



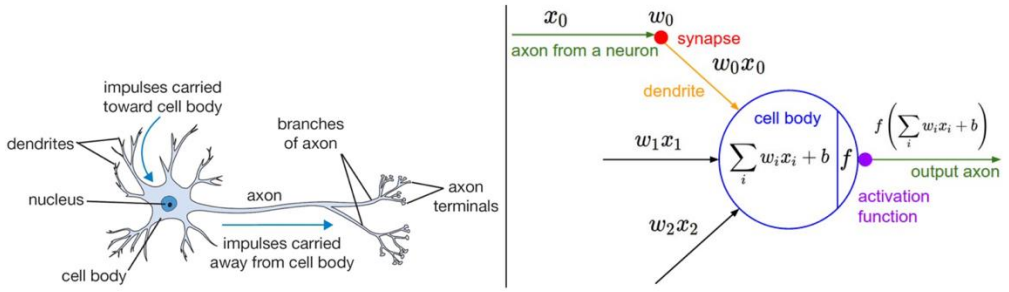
- ReLU (Rectified Linear Unit)



artificial neuron (node in neural network):

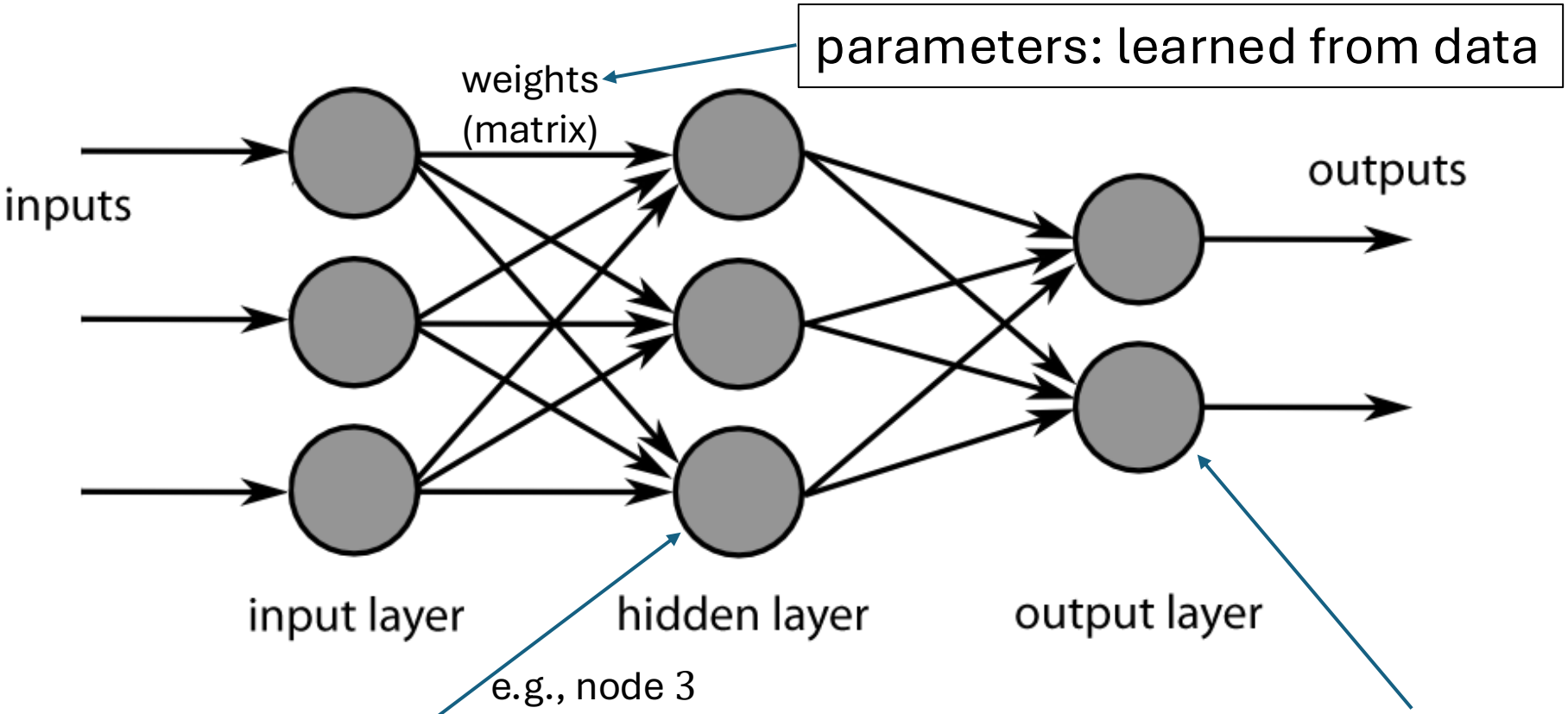


inspired from biological neurons:

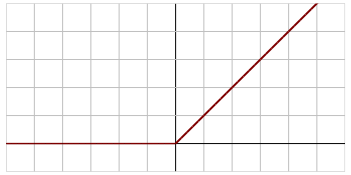


source

Neural Network as Nonlinear Function



linear transformation with nonlinear activation functions

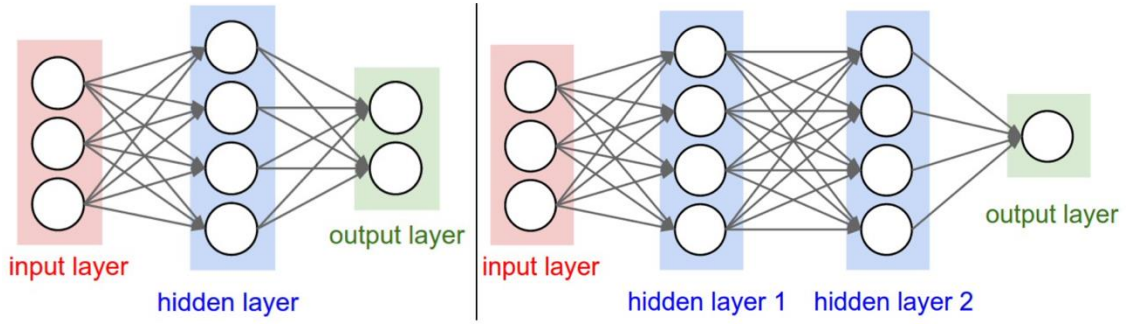


$$\left(\boldsymbol{w}_3 \cdot \boldsymbol{x} \right)$$

Multi-Layer Perceptron (MLP)

fully-connected feed-forward network with at least one hidden layer (universal function approximator)

binary classification



regression

toward deep learning: add hidden layers

more layers (depth) more efficient than just more nodes (width): less parameters needed for same function complexity

Classification Networks

cross-entropy loss (one output node for each class k):

$$L(y, \hat{y}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

using softmax on output nodes:

$$\hat{y}_k = \frac{e^{o_k}}{\sum_{l=1}^{K} e^{o_l}}$$

→ prediction of probabilities

Regression Networks

squared error loss (just one output node):

$$L(y, \hat{y}) = (y - \hat{y})^2$$

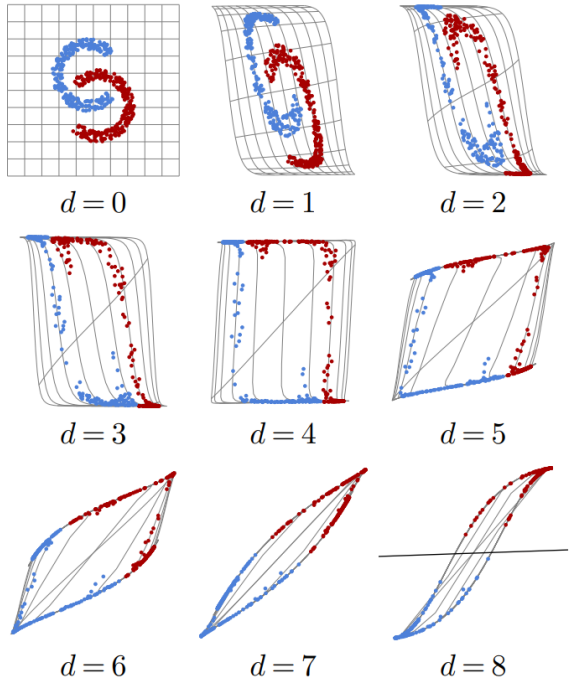
no function (identity function) on output

→ prediction of real numbers

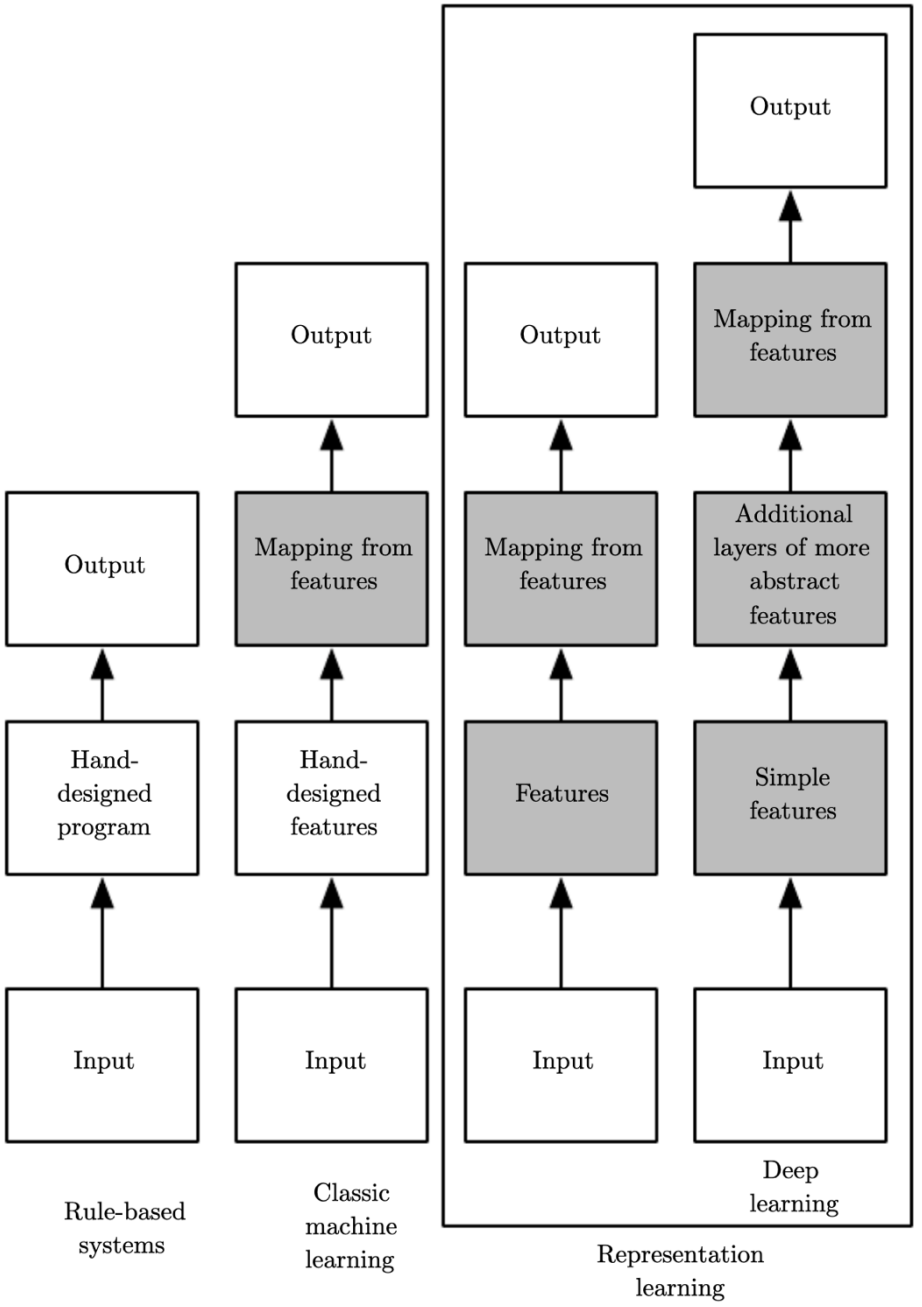
Ladder of Generalization

classic ML: feature engineering

deep learning: feature learning



source



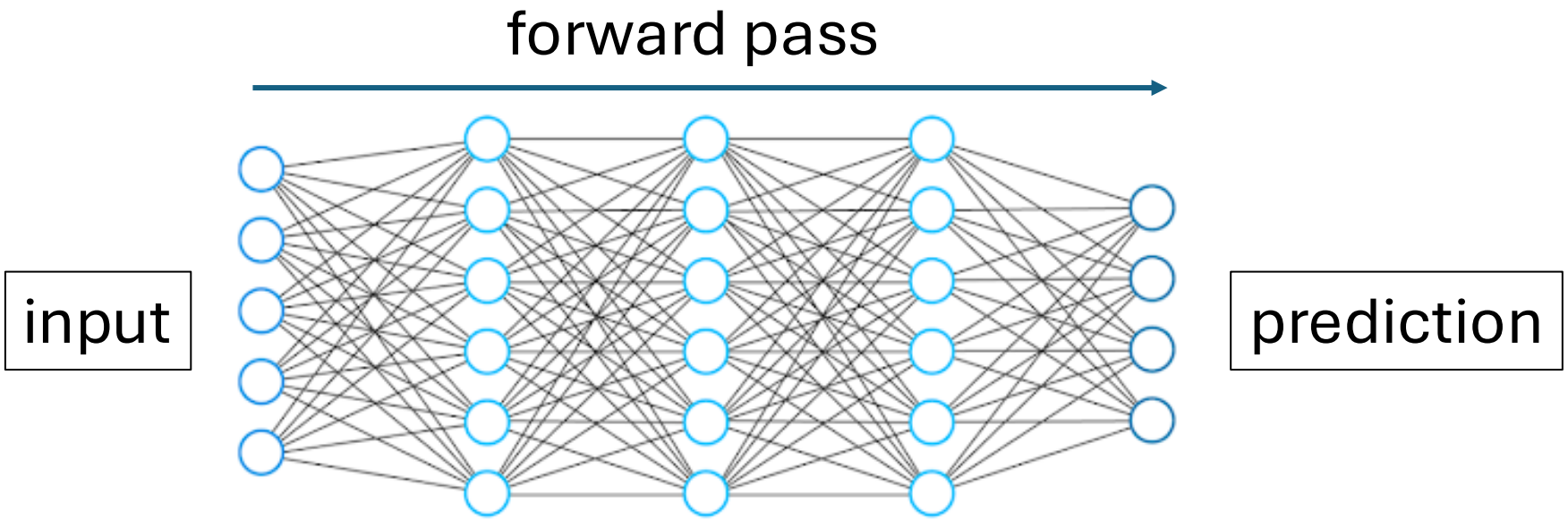
source

Network Training

Forward Pass

for each training example:

- fix current weights
- compute prediction



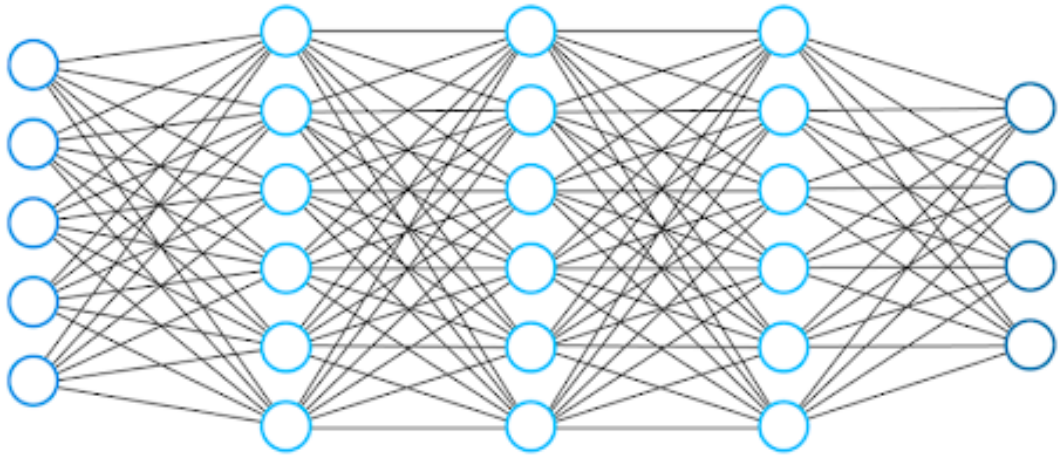
Backward Pass

cost function: $J(\boldsymbol{w}) = \frac{1}{n}\sum_{i=1}^{n} L_i(\boldsymbol{w})$

gradient: $\nabla_{\boldsymbol{w}} J(\boldsymbol{w})$

for all training examples (in considered batch):

- compute individual errors (losses)
- calculate gradient: backpropagation via chain rule of calculus



backward pass

predictions

= ? ← target

error measurement

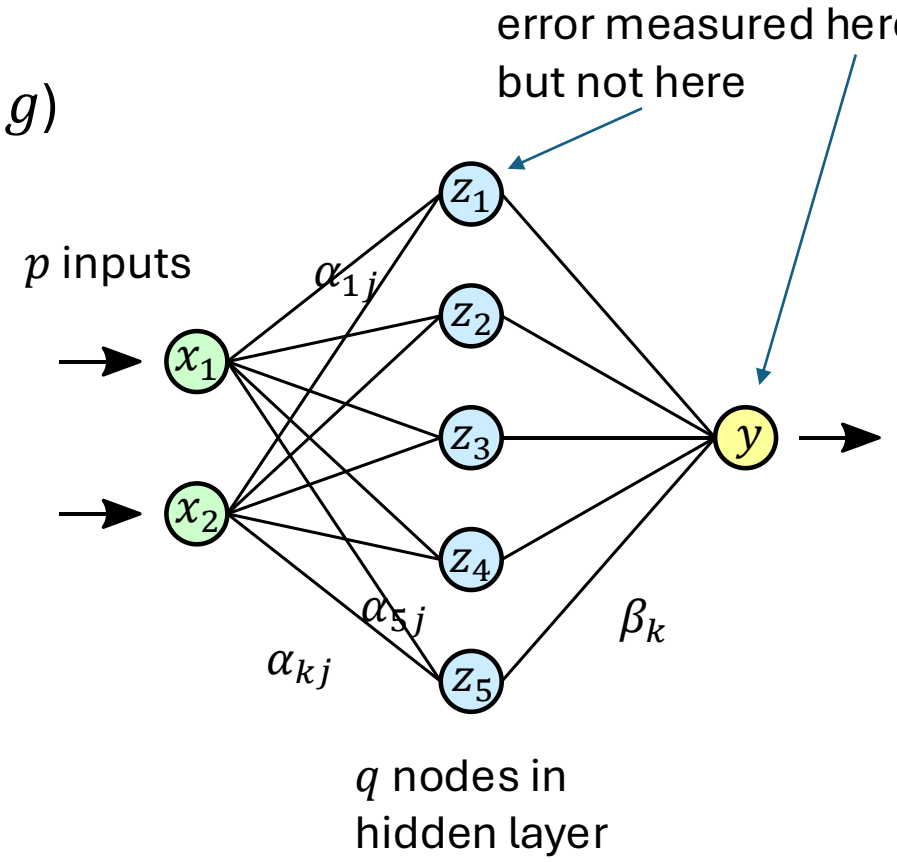
Example WOLOG

- regression (squared error loss, identity output function g)
- with one hidden layer ($\boldsymbol{w}$: $\boldsymbol{\alpha}, \boldsymbol{\beta}$)

$$\hat{y}_i = f(\boldsymbol{x}_i; \boldsymbol{w}) = g(\boldsymbol{z}_i; \boldsymbol{\beta}) = \sum_{k=0}^{q} \beta_k z_{ik}$$

$$z_{ik} = h(\boldsymbol{x}_i; \boldsymbol{\alpha}_k) = h\left(\sum_{j=0}^{p} \alpha_{kj} x_{ij}\right)$$

activation function



cost function:

$$J(\boldsymbol{w}) = \sum_{i=1}^{n} L_i(y_i, \hat{y}_i; \boldsymbol{w}) = \sum_{i=1}^{n} \left(y_i - f(\boldsymbol{x}_i;\ \boldsymbol{\alpha}, \boldsymbol{\beta})\right)^2$$

Example WOLOG

gradients:

$$\frac{\partial L_i}{\partial \beta_k} = -2\,(y_i - \hat{y}_i) z_{ik} = \delta_i z_{ik}$$

$$\frac{\partial L_i}{\partial \alpha_{kj}} = -2\,(y_i - \hat{y}_i)\beta_k h_k' \left(\sum_{j=0}^{p} \alpha_{kj} x_{ij} \right) x_{ij} = s_{ik} x_{ij}$$

backpropagation equations: $s_{ik} = h_k' \left(\sum_{j=0}^{p} \alpha_{kj} x_{ij} \right) \beta_k \delta_i$

use errors of later layers to calculate errors of earlier ones (avoiding redundant calculations of intermediate terms)

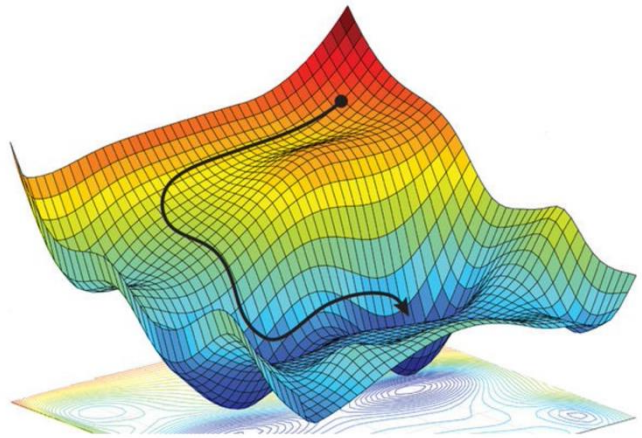
Using Gradients for Iterative Learning

use gradients found via backpropagation to iteratively update weights (gradient descent):

$$\beta_k^{(r+1)} = \beta_k^{(r)} - \eta_r \sum_{i=1}^{n} \frac{\partial L_i}{\partial \beta_k^{(r)}} \qquad \alpha_{kj}^{(r+1)} = \alpha_{kj}^{(r)} - \eta_r \sum_{i=1}^{n} \frac{\partial L_i}{\partial \alpha_{kj}^{(r)}}$$

- adaptive learning rate η_r: learning rate often adjusted per iteration
- weight initialization: choose small random weights as starting values to break symmetry (typically using some heuristics)

Stochastic Gradient Descent (SGD)



step size

cost function

weight updates: $\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \nabla_{\boldsymbol{w}} \mathcal{L}$

options:

all training examples

- $\mathcal{L} = \frac{1}{n} \sum_{i=1}^{n} L_i$ batch gradient descent (whole training dataset)
- $\mathcal{L} = L_i$ stochastic gradient descent (single example)
- $\mathcal{L} = \frac{1}{m} \sum_{i=1}^{m} L_i$ mini-batch stochastic gradient descent

mini-batch size → hyperparamter: tradeoffs between runtime & memory, convergence & generalization

random fluctuations of SGD help to escape saddle points

Training: Iterative Adjustment of Weights

```
for epoch in range(epochs):
    for X, y in mini_batches:
        pred = model(X)
        cost = loss(pred, y)
        cost.backward()
        optimizer.step()
```

forward pass

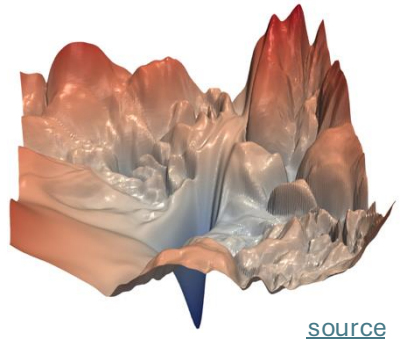
quantification of error

backpropagation

gradient descent: one step in direction of steepest descent

How to Train Deep Neural Networks Effectively?

typical loss surface:



source

optimization and regularization difficult

- local vs global minima, saddle points, easily overfitting
- many hyperparameters to tune

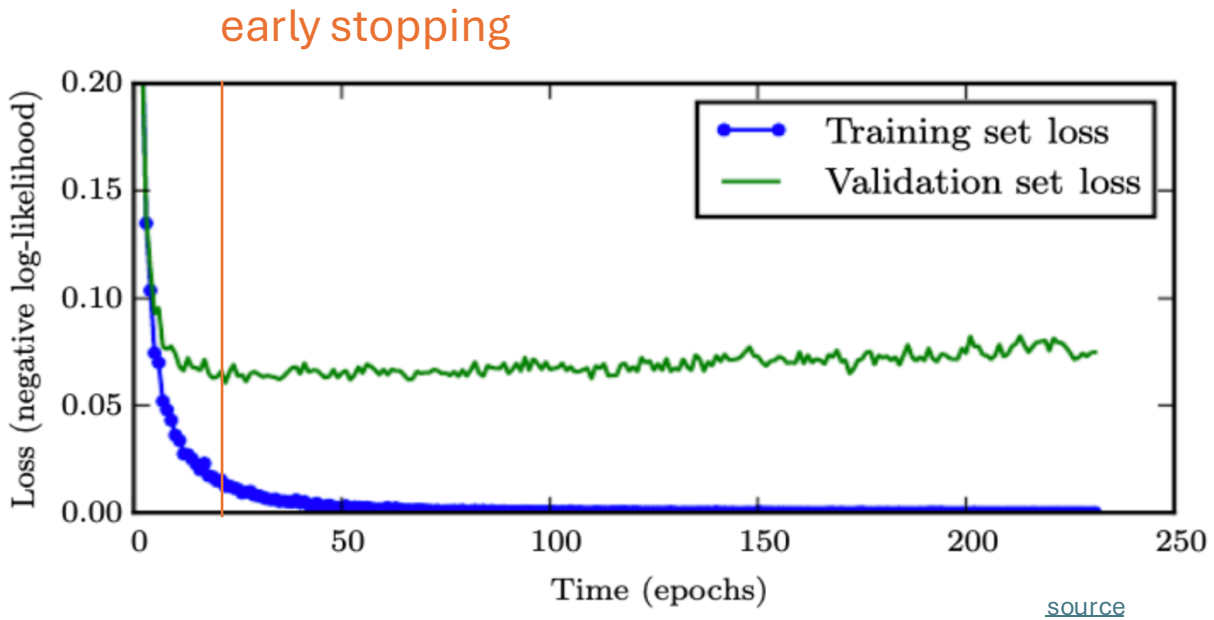
but many methods to get it working in practice (despite partly patchy theoretical understanding)

- optimization: activation and loss functions, weight initialization, adaptive learning rate (e.g., Adam), learning rate schedulers
- explicit regularization: weight decay, dropout, data augmentation
- implicit regularization: early stopping, batch/layer normalization, SGD

Early Stopping

loss independently measured on validation set

halting training when overfitting begins to occur



source

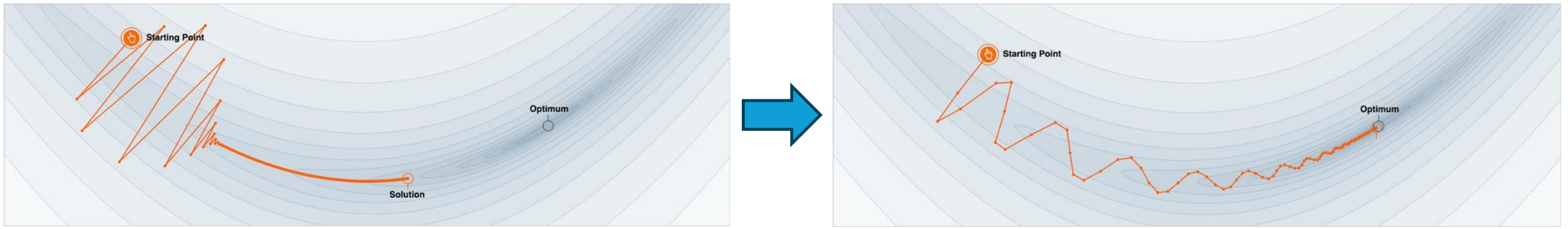
Gradient Descent with Momentum

algorithm:

- estimate gradient: $\boldsymbol{g} = \nabla_{\boldsymbol{w}} \mathcal{L}$
- compute velocity update: $\boldsymbol{v} \leftarrow \alpha\, \boldsymbol{v} - \eta\, \boldsymbol{g}$
- apply weight update: $\boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{v}$

hyperparameter ($0 < \alpha < 1$) specifying exponential decay

→ no bouncing around of weights, escape from local minima and saddle points



Adaptive Learning Rate

better convergence by adapting learning rate for each weight per iteration: lower/higher learning rates for weights with large/small updates

popular methods (g_w denotes component of gradient for **individual weight** w):

- Adagrad: $w \leftarrow w - \frac{\eta}{\sqrt{\sum_{\tau=1}^{t} g_{w,\tau}^2}} g_w$ with t, τ denoting current and past iterations

  (issue: sum in denominator grows with more iterations → can get stuck)
- RMSProp: $w \leftarrow w - \frac{\eta}{\sqrt{v(w)}} g_w$ with $v(w) \leftarrow \gamma\, v(w) + (1-\gamma) g_w^2$
- Adam (Adaptive Moment Optimization): combines RMSProp with momentum

Learning Rate Schedulers

learning rate η in gradient descent typically set to small value (e.g., 0.001)

use momentum and adaptive learning rates to improve optimization

learning rate scheduling as further alternative (can be used on top):

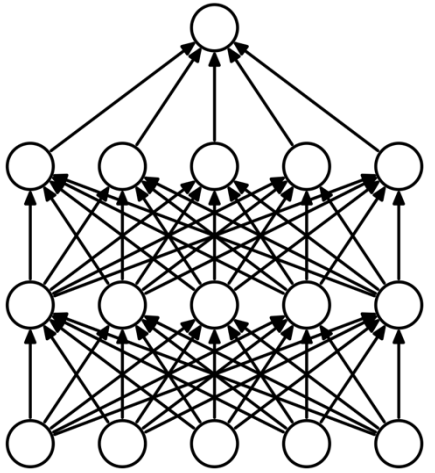
- decay by a certain factor every given number of epochs
- reduce when a metric has stopped improving
- smooth decay by cosine annealing (optionally cyclic with restarts)
- …

Dropout in Neural Networks

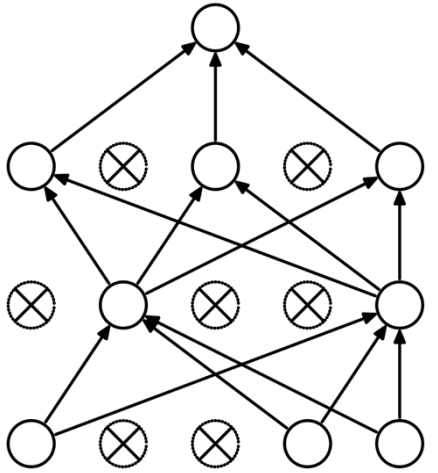
goal: prevent overfitting of large neural networks

randomly drop non-output nodes (along with their connections) during training (not prediction)

→ adaptability: regularizing each hidden node to perform well regardless of which other hidden nodes are in the model



(a) Standard Neural Net



(b) After applying dropout.

source

- for each mini-batch, randomly sample independent binary masks for the nodes
- destroying extracted features rather than input values

Batch Normalization

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

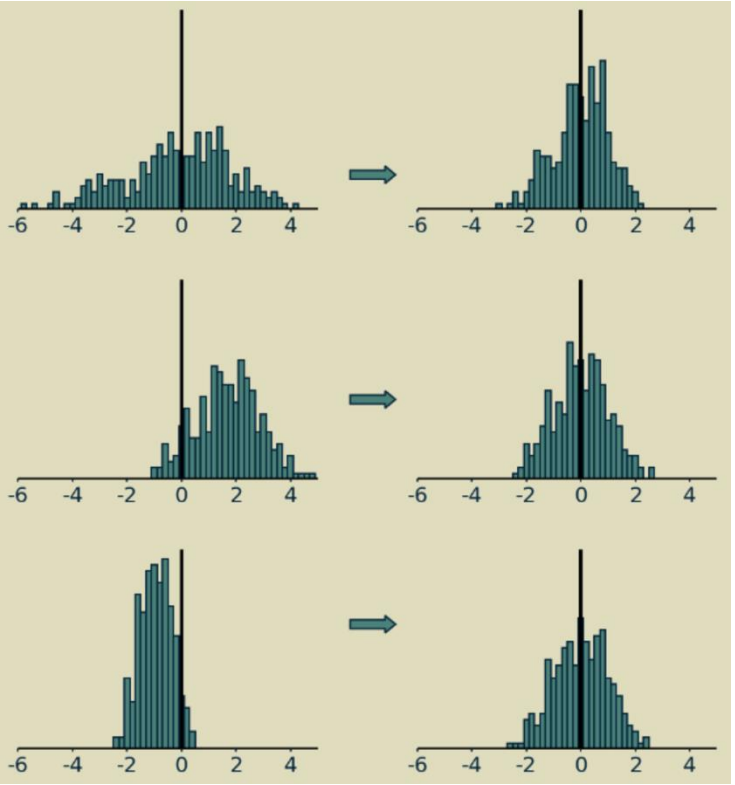
$$\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma\widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

source



source

adaptive reparameterization of inputs to network layer (before or after activation)

independently for each feature

(implicit) regularization effect

Advanced Network Architectures

From Permutation Invariance to Structured Deep Learning

fully-connected networks treat input features as unordered

(e.g., the different pixel intensities in an image)

but real data is often structured → local correlations between features

- images**:** nearby pixels are highly correlated
- text / speech: words and sounds depend on previous context
- time series: observations depend on earlier time steps

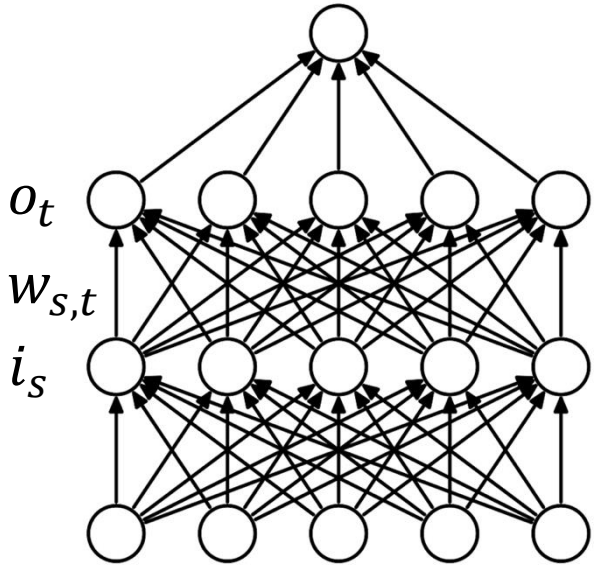
ignoring these structures leads to suboptimal models → CNNs, RNNs

Structure of Fully-Connected Neural Networks

computation in fully-connected network:

matrix multiplication of scalar inputs and weights

$$o_t = \sum_s w_{s,t}\, i_s$$



dropped dimension of different training observations (in mini-batch) in this view

Convolutional Neural Networks (CNN or ConvNet)

actually, correlation (convolution would have – here):

$$o_{x,y} = \sum_{s,t} w_{s,t}\ i_{x+s,y+t}$$

feature map (transformed image) — $o_{x,y}$

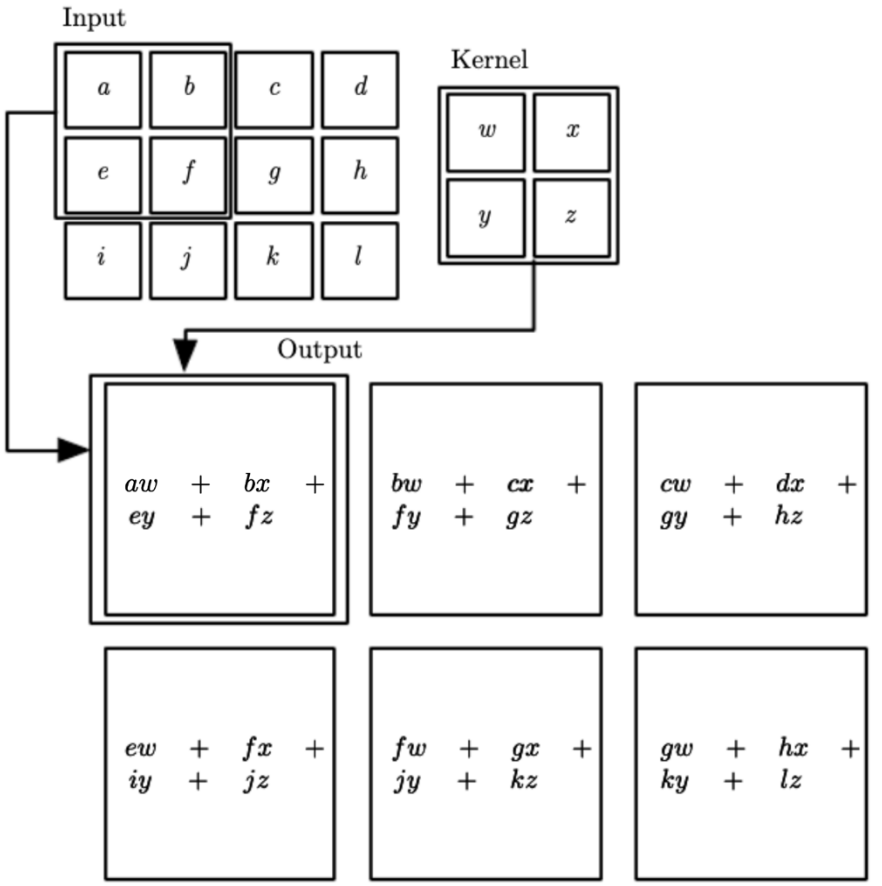
kernel (learned) — $w_{s,t}$

image (input or feature map) — $i_{x+s,y+t}$

(again, dropping dimension of different training observations)

→ exploit local spatial correlations

2D convolution

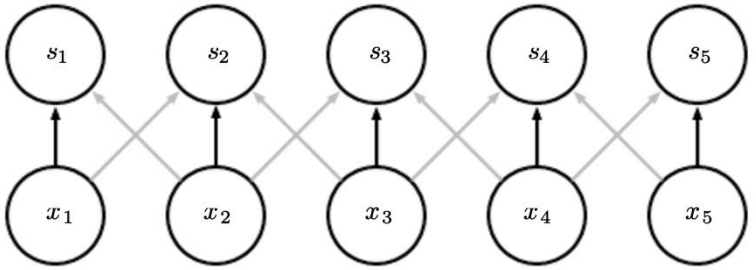


source

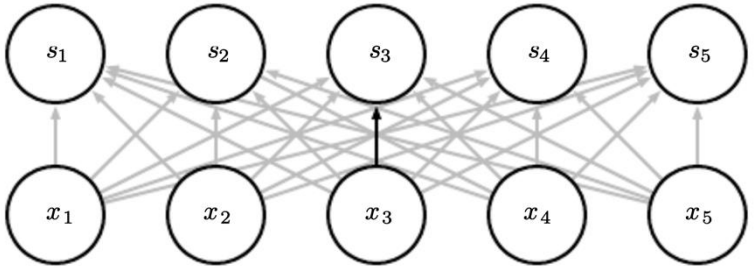
Regularization Effects

- sparse interactions: much less weights
- parameter sharing: use same weights for different connections (locations)

convolutional layer:



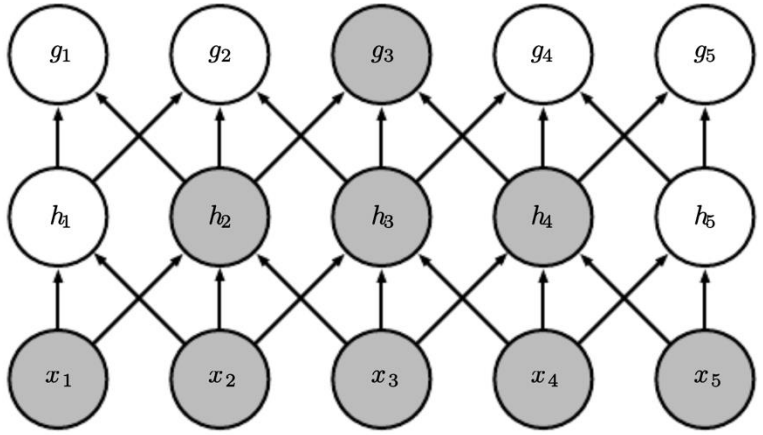
fully-connected layer:



effect of receptive field over several layers:

- consider only locally restricted number of input values from previous layer
- grows for earlier layers (indirect interactions)

→ hierarchical patterns from simple building blocks (many aspects of nature hierarchical)



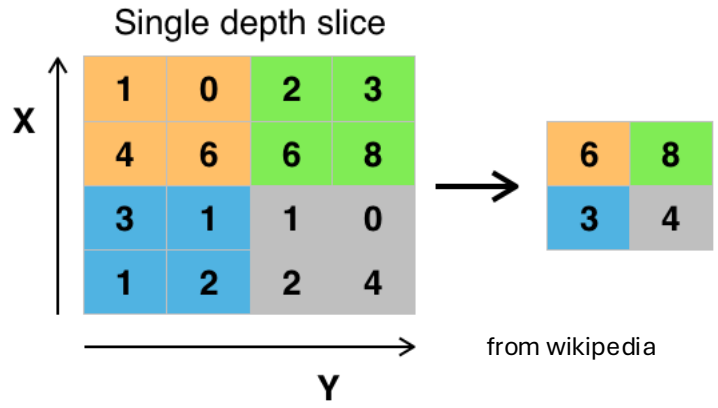
source

Another CNN Ingredient: Pooling

replacing outputs of neighboring nodes with summary statistic (e.g., maximum or average value of nodes)

→ non-linear downsampling

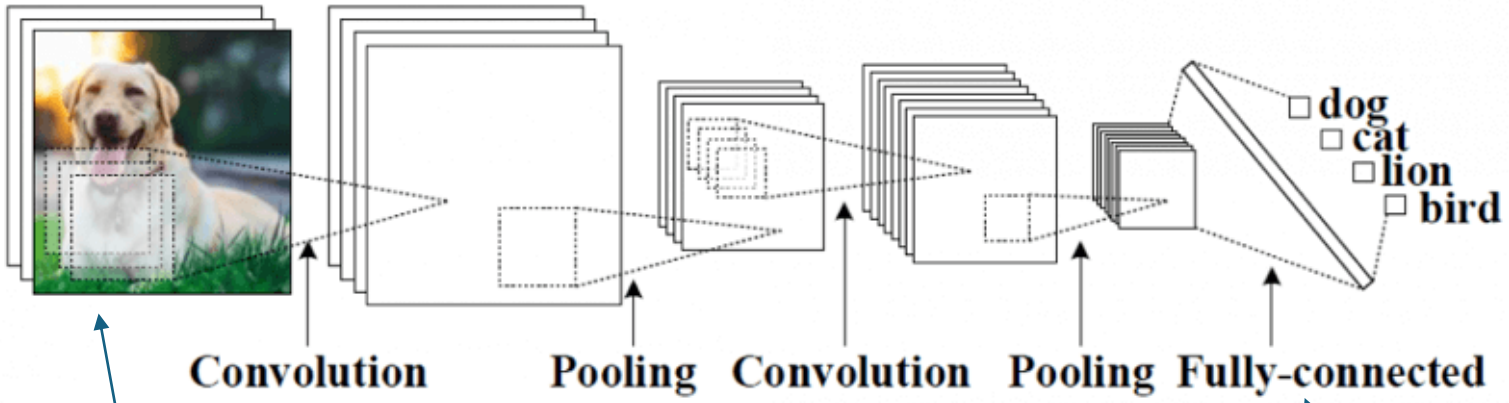
max pooling:



pooling is translation invariant: no interest in exact position of, e.g., maximum value

Putting It All Together

convolutional neural networks in short:
local connections, shared weights, pooling, many layers

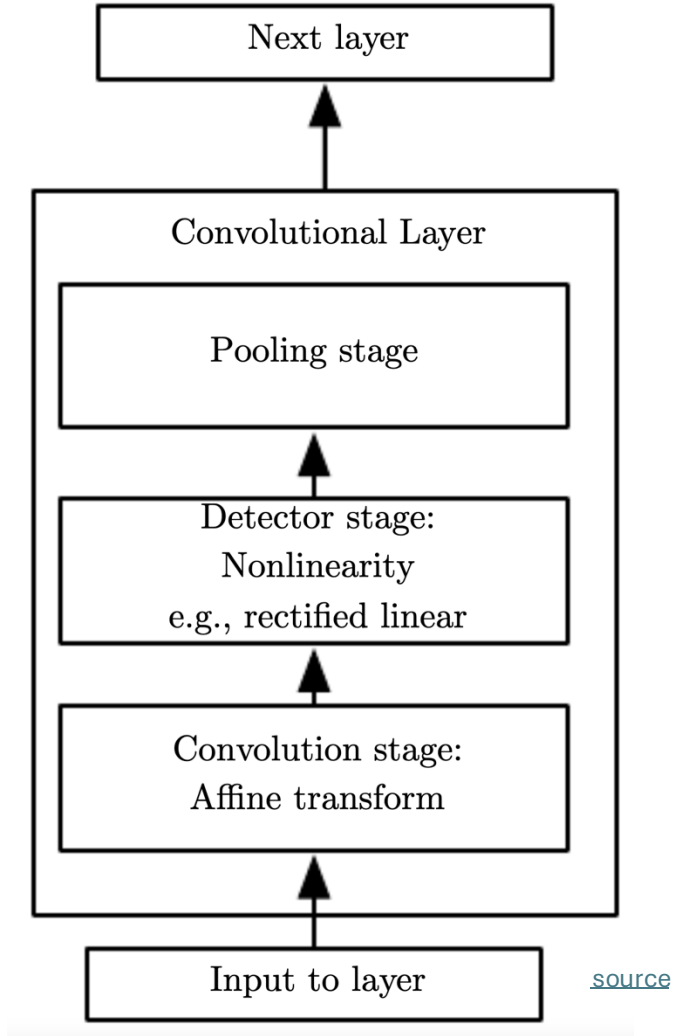


many images (training examples), potentially with several channels

several kernels producing several feature maps

downsampling

flatten dimensions for final classification

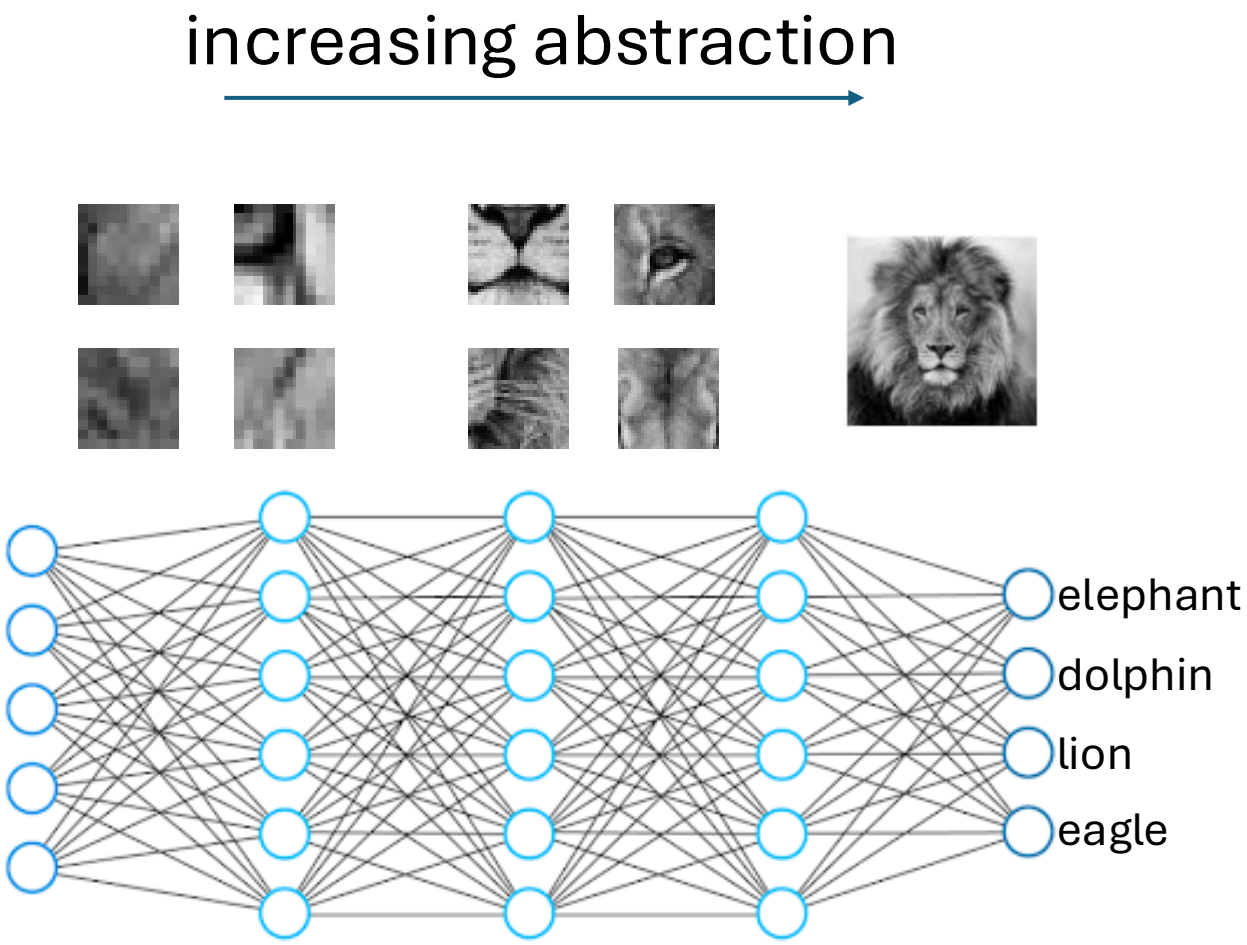


source

Hierarchical Representation

deep learning:
several hidden layers

hierarchy of concepts learned from raw data in deep graph with many layers

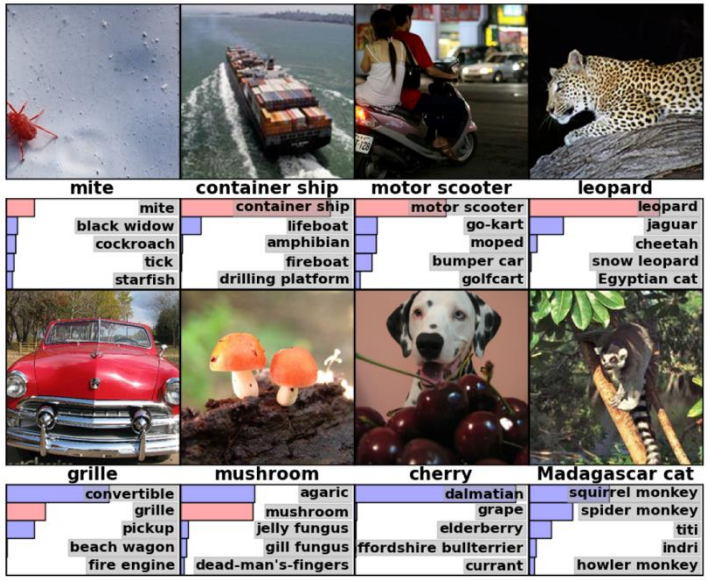


Rise of Deep Learning

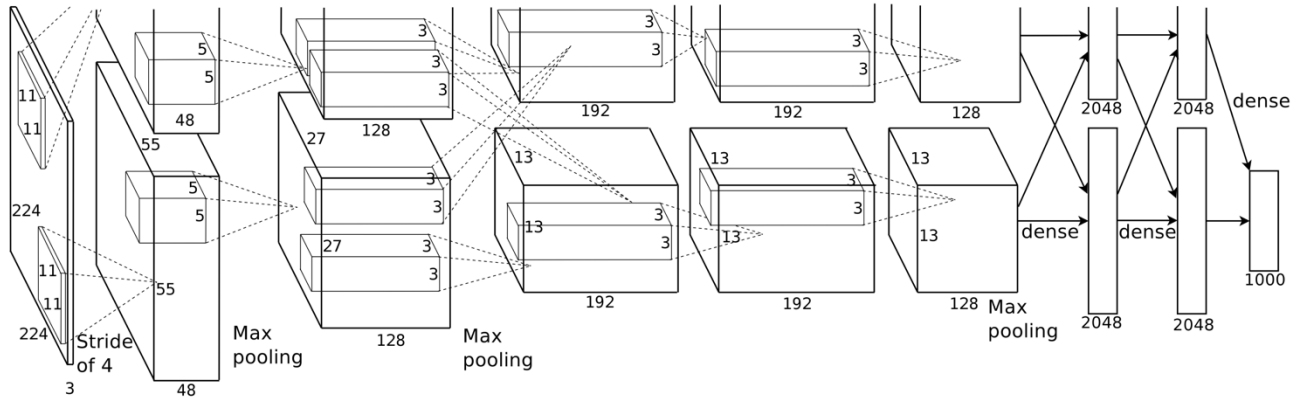
a little bit oversimplified:

deep learning = lots of training data + parallel computation + smart algorithms

AlexNet: ImageNet (with data augmentation) + GPUs + ReLU, dropout, SGD



source



Transfer Learning

problem with ConvNets trained from scratch (in fact, with all ML methods): only suited for domain of training data

idea of transfer learning:

pretrain a big model (called foundation model) on a broad data set, and then use these learnings for subsequent trainings on specific (typically, narrow) data

two forms of transfer learning: feature extraction and finetuning

Feature Extraction

approach:

- get a pretrained ConvNet as foundation model
- remove final fully-connected layer (its outputs are the class scores from the pretraining task)
- pass training data (for the task at hand) through the network
- for each image, extract vector of last hidden layer‘s activations (e.g., 4096-dimensional for AlexNet)
- use the extracted values as features to train another model (to be adjusted to the task at hand)

(same idea as used before for SIFT features)

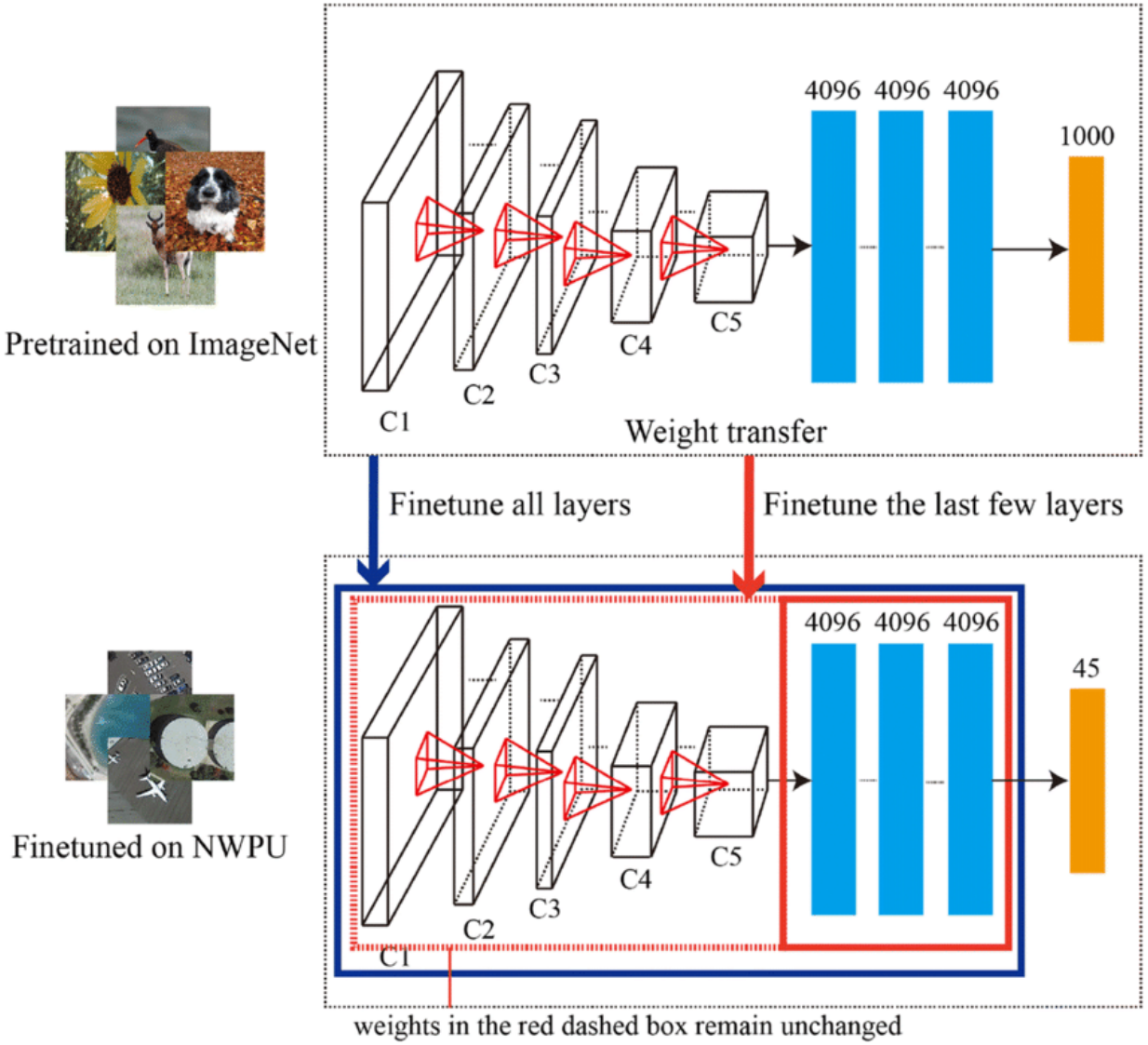
Finetuning

approach:

- get pretrained ConvNet as foundation model
- continue (starting with weights after pretraining) training with data for task at hand

typically, need to change architecture of final layer: different number of classes

options: finetune all weights or just some



source

Finetuning Scenarios

finetuning gives better adjustment to new task than feature extraction (important if new task differs a lot from pretraining task)

→ coming along with danger of overfitting

ConvNet features more generic in early layers (e.g., edges) and more specific to the data set of pretraining in later layers

→ for small finetuning data sets, typically keep weights of early layers fixed to avoid overfitting

for large finetuning data sets (less danger of overfitting), finetuning all layers can be beneficial

Recurrent Neural Networks (RNN)

speech recognition, natural language processing, time series, …

problem: need to generalize across different points in time (or multiple positions of words within a sentence) and learn from context

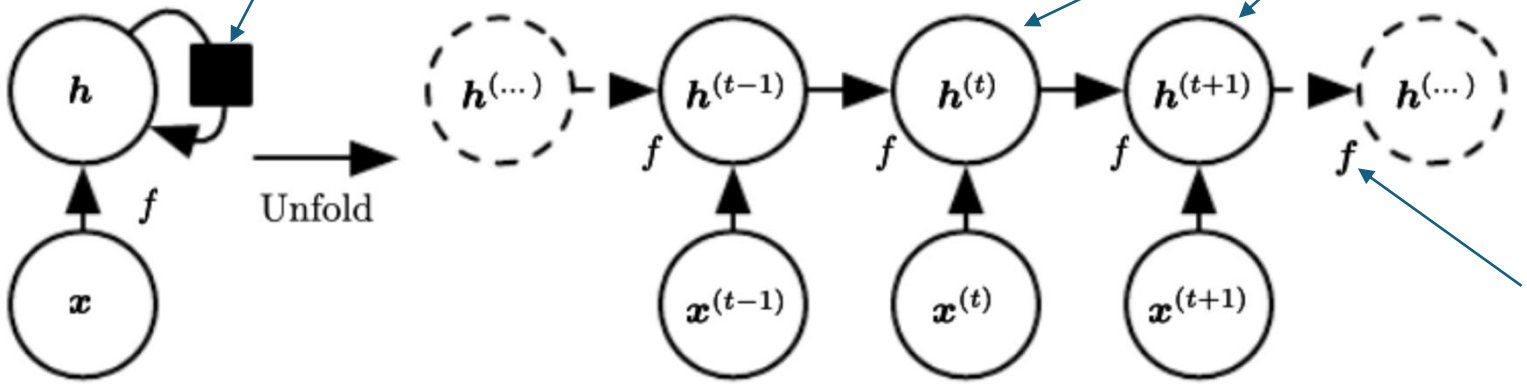
idea: parameter sharing across different parts of model

CNNs apply parameter sharing (convolutions) on grid-like structures (including 1-D grids like time series or speech), but this is limited to neighboring inputs (per layer).

→ need for approach to learn sequential structures (process input as stream, e.g., speech, rather than one batch, e.g., image)

Back-Propagation through Time

RNN: output of node directed back to input



interaction with delay of single time step

activation of recurrent nodes at that point in time

computational graph with repeated pieces:

hidden layer: several recurrent neurons

input layer: e.g., one-hot encoded words or embeddings

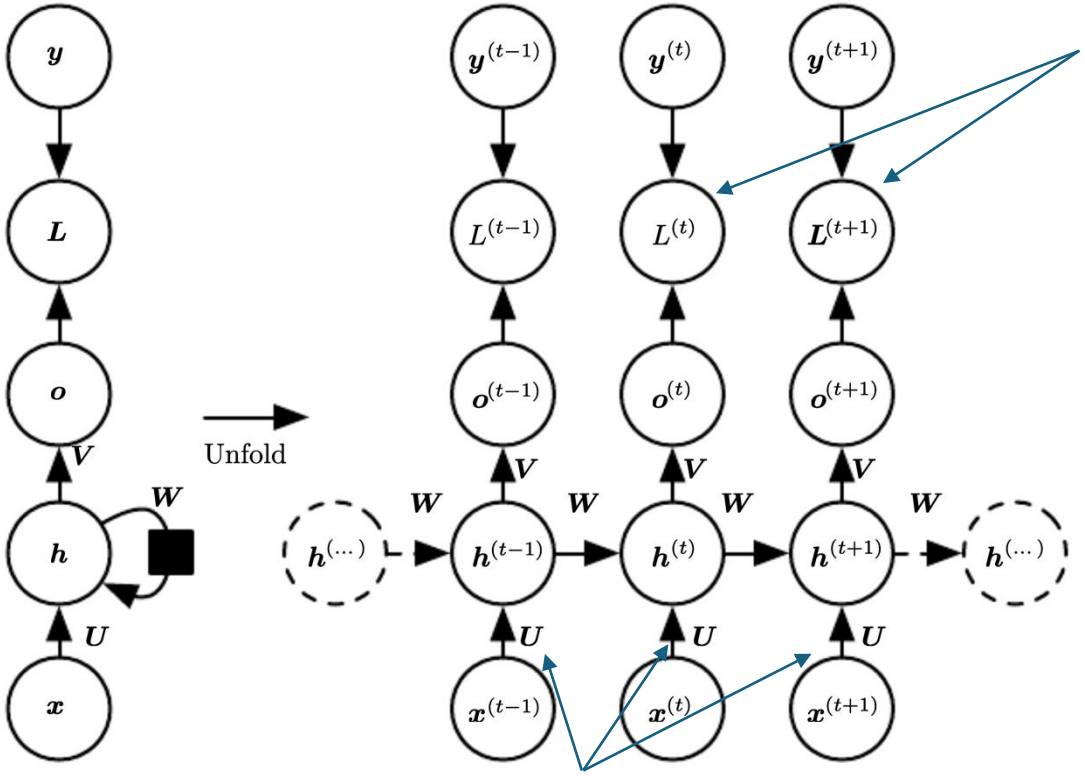
single model operating on all time steps

source

different kind of depth: one recurrence for each sequential step (e.g., word)

Weight Sharing across Time

example: input-output mapping at each time step with recurrent hidden nodes



parameter/weight sharing over time

source

total loss for sequence: sum of losses over time steps

$$
\begin{aligned}
\boldsymbol{a}^{(t)} &= \boldsymbol{b} + \boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{U}\boldsymbol{x}^{(t)}, \\
\boldsymbol{h}^{(t)} &= \tanh(\boldsymbol{a}^{(t)}), \\
\boldsymbol{o}^{(t)} &= \boldsymbol{c} + \boldsymbol{V}\boldsymbol{h}^{(t)}, \\
\hat{\boldsymbol{y}}^{(t)} &= \operatorname{softmax}(\boldsymbol{o}^{(t)}),
\end{aligned}
$$

Markov dependence on previous hidden state

more popular than ReLU here (exploding gradients)

discrete outputs

Further Examples

summarization of sequence:

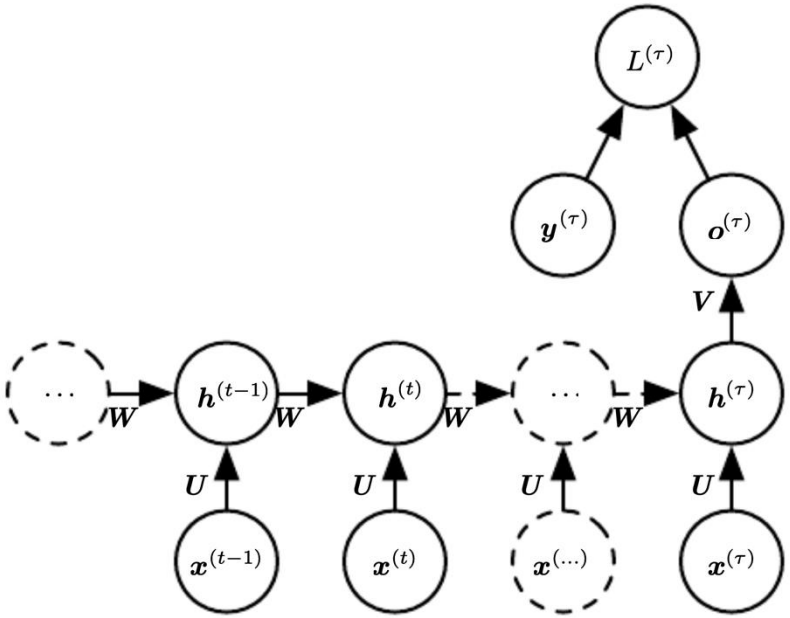
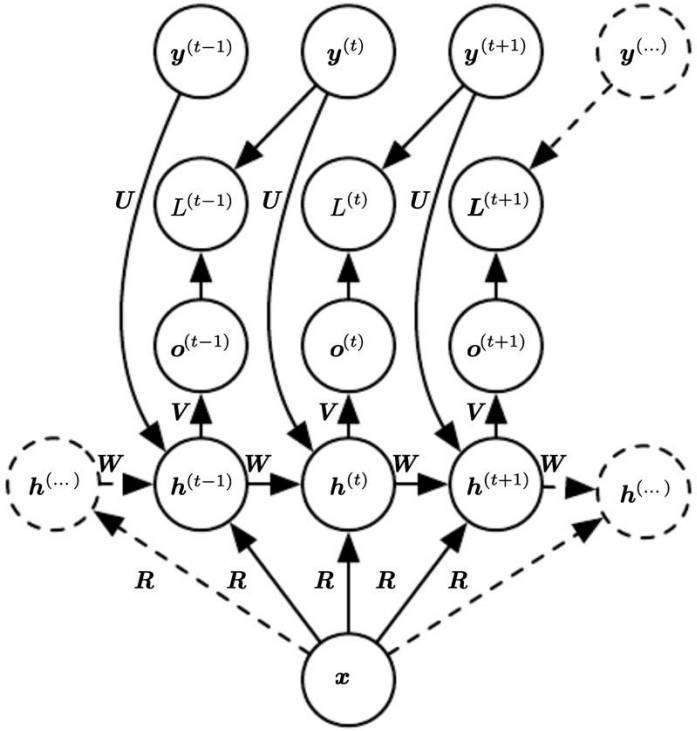
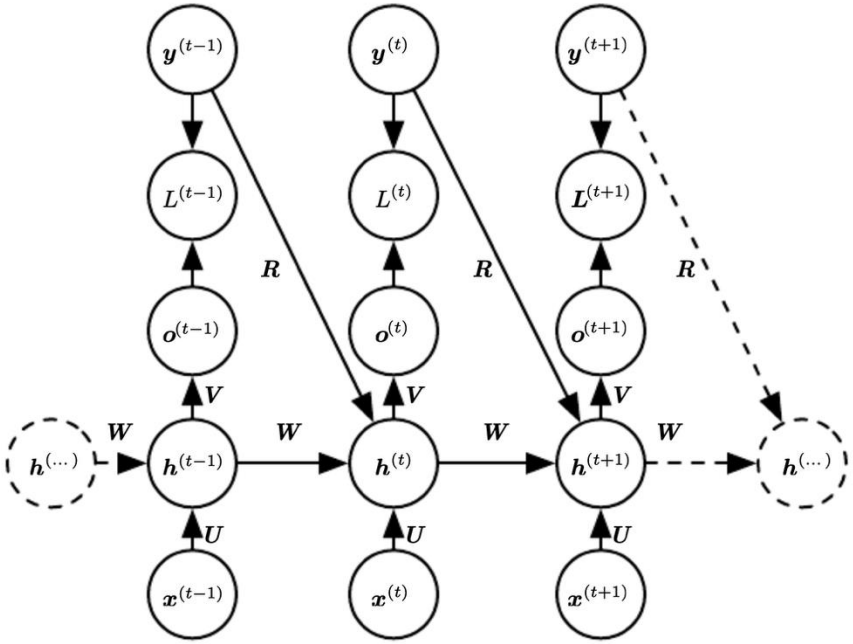


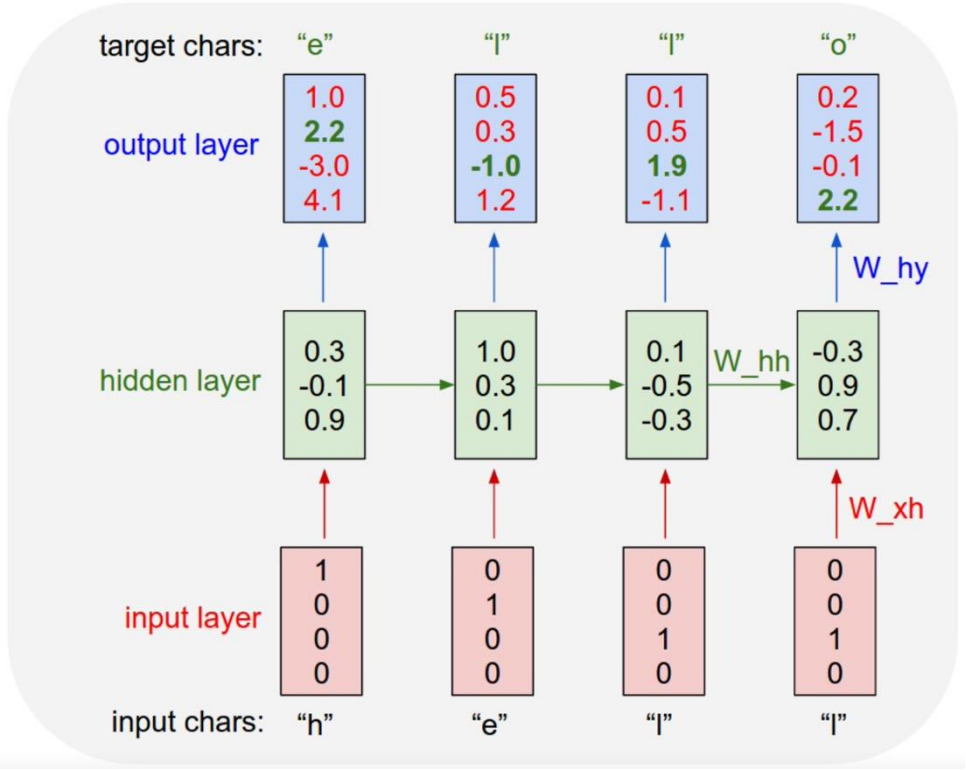
image captioning:



conditional sequence:

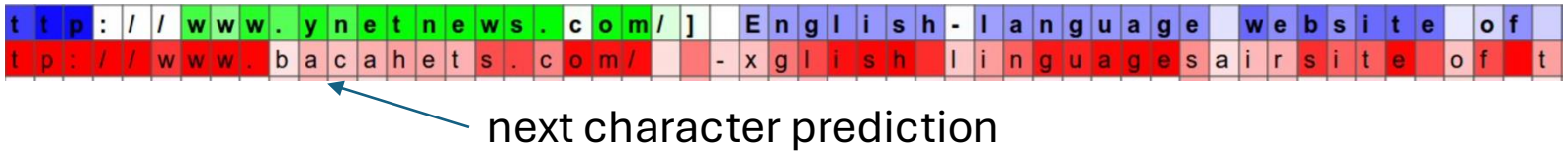


Visualization



source

neuron getting excited inside URLs (excited, not excited):



high/low activation:

Cell sensitive to position in line:

```
The sole importance of the crossing of the Berezina lies in the fact
that it plainly and indubitably proved the fallacy of all the plans for
cutting off the enemy's retreat and the soundness of the only possible
line of action--the one Kutuzov and the general mass of the army
demanded--namely, simply to follow the enemy up. The French crowd fled
at a continually increasing speed and all its energy was directed to
reaching its goal. It fled like a wounded animal and it was impossible
to block its path. This was shown not so much by the arrangements it
made for crossing as by what took place at the bridges. When the bridges
broke down, unarmed soldiers, people from Moscow and women with children
who were with the French transport, all--carried on by vis inertiae--
pressed forward into boats and into the ice-covered water and did not,
surrender.
```

Cell that turns on inside quotes:

```
"You mean to imply that I have nothing to eat out of.... On the
contrary, I can supply you with everything even if you want to give
dinner parties," warmly replied Chichagov, who tried by every word he
spoke to prove his own rectitude and therefore imagined Kutuzov to be
animated by the same desire.

Kutuzov, shrugging his shoulders, replied with his subtle penetrating
smile: "I meant merely to say what I said."
```

Cell that robustly activates inside if statements:

```
static int __dequeue_signal(struct sigpending *pending, sigset_t *mask,
        siginfo_t *info)
{
  int sig = next_signal(pending, mask);
  if (sig) {
    if (current->notifier) {
      if (sigismember(current->notifier_mask, sig)) {
        if (!(current->notifier)(current->notifier_data)) {
          clear_thread_flag(TIF_SIGPENDING);
          return 0;
        }
      }
    }
    collect_signal(sig, pending, info);
  }
  return sig;
}
```

long-term memory: weights
short term memory: activation patterns (context)

Gated RNNs

issue with long chain of gradients through recurrences: vanishing (mainly) and exploding gradients in back-propagation

→ need to focus on important sequence elements

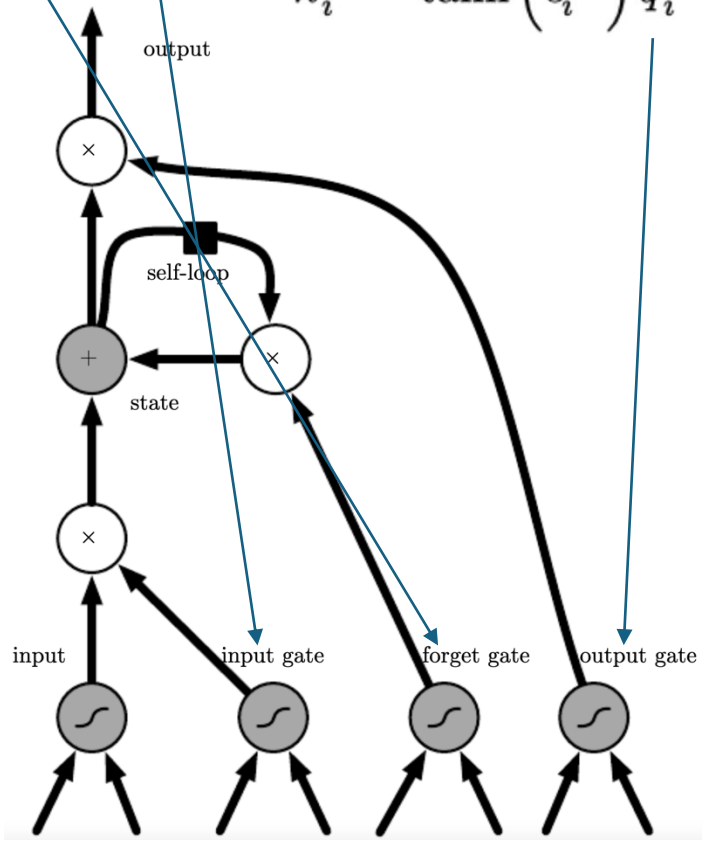
replace usual recurrent hidden nodes with long short-term memory (LSTM) cells with internal recurrence (self-loop)

- linear self-loop in addition to outer recurrence of RNN: error carousel preserving (→ long) short-term memory (i.e., activation patterns)
- sigmoid activations with independent weights for input, forget, and output gates

other prominent gated RNN: gated recurrent unit (GRU)

$$s_i^{(t)} = f_i^{(t)} s_i^{(t-1)} + g_i^{(t)} \sigma\left(b_i + \sum_j U_{i,j} x_j^{(t)} + \sum_j W_{i,j} h_j^{(t-1)}\right)$$

$$h_i^{(t)} = \tanh\left(s_i^{(t)}\right) q_i^{(t)}$$



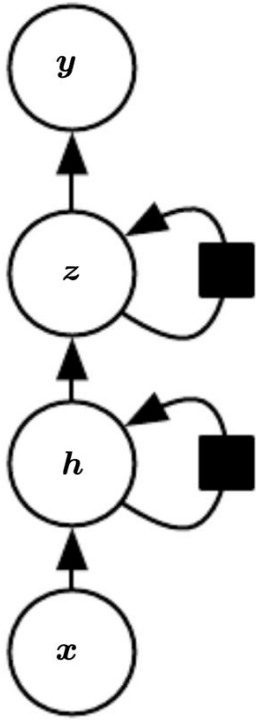
source

Issues with RNNs

- especially LSTMs very computationally expensive (with lots of parameters)
- transfer learning (using pre-trained network layers on new task, e.g., popular for CNNs) difficult

(hierarchical) representation learning: go deep by stacking several layers of recurrent nodes (e.g., important for speech recognition) → worsening efficiency even more

(self-)attention and transformers to the rescue …



source

Tabular vs Unstructured Data

Deep learning excels on unstructured data (images, text, audio), but not necessarily on tabular data.

challenges of tabular data for deep learning:

- heterogeneous features (mixed numerical, categorical, sparse variables)
- limited benefit from feature learning → feature engineering often still required
- weak transfer learning / pretraining opportunities

Tree-based methods (e.g., gradient boosting) naturally handle these properties and often perform better on tabular tasks.